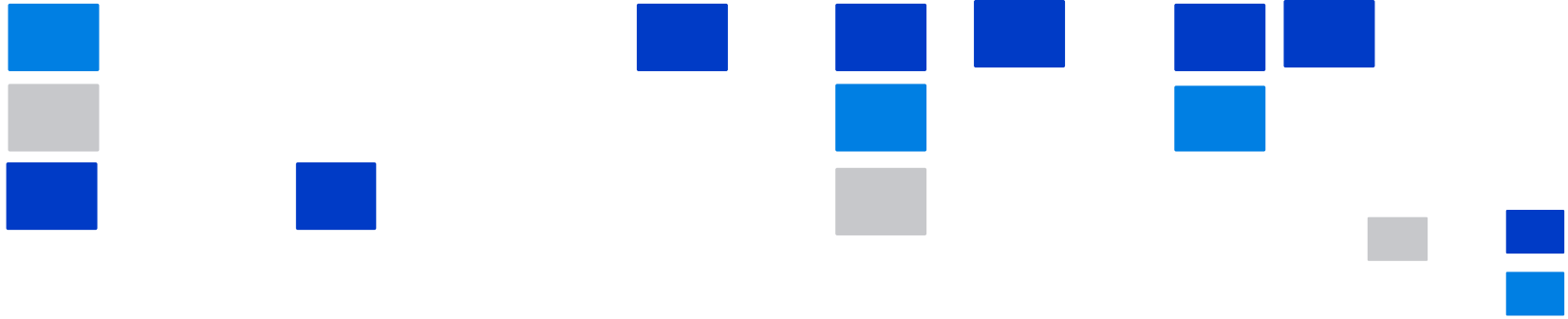




# Sistemes Operatius

Tema 4: Crides al sistema - Pipes • Curs 2026-27

# Pipes

**Pipe:** el primer mecanisme de comunicació de processos que va aparèixer. Està format per 2 file descriptors, un serveix per llegir (0) i l'altre per escriure (1).

- Conjunt de bytes sense format.
- Lectures destructives.
- Les lectures són bloquejants.
- Les pipes són unidireccionals.
- Tenen una capacitat limitada (64 KB).
- És important tancar els file descriptors que no s'usen d'una pipe (`close()`)

# Duplicació i redirecció: dup() i dup2()

- **dup (oldfd)** retorna el descriptor lliure amb el número més baix i el fa referir a la mateixa descripció d'arxiu oberta que `oldfd`.
- dup2 (oldfd, newfd)** redirigeix un descriptor concret. Si `newfd` estava obert, el tanca de manera atòmica abans de reutilitzar-lo.

Patró habitual per enviar la sortida estàndard a una pipe:

```
dup2 (pipefd[1], STDOUT_FILENO);  
close (pipefd[0]); close (pipefd[1]);
```

Després de la redirecció, les escriptures a **STDOUT\_FILENO** arriben a la pipe.

# Execució amb la família `exec()`

- La família **`exec()`** substitueix el programa del procés actual per un altre executable; no crea cap procés nou.

Si té èxit, no retorna. Si falla, retorna `-1` i estableix **`errno`**.

Exemple després de preparar les redireccions:

```
execlp("sort", "sort", NULL);
```

El patró **`fork()`** + **`dup2()`** + **`exec()`** permet connectar programes mitjançant pipes.

# Example

```
int main() {  
    int fd[2];  
    char buffer[256];  
    int nbytes, pid;  
  
    if (pipe(fd) == -1) {  
        write(STDERR_FILENO, "Error creating pipe\n", 20);  
        exit(-1);  
    }
```

# Exemple

```
pid = fork();
switch (pid) {
    case -1: // Error forking
        write(STDERR_FILENO, "Error forking\n", 14);
        close(fd[0]);
        close(fd[1]);
        exit(-1);
    case 0: // Child
        close(fd[0]);
        write(fd[1], "Pipe message\n", 13);
        close(fd[1]);
        exit(0);
    default: // Parent
        close(fd[1]);
        nbytes = read(fd[0], buffer, 256);
        write(STDOUT_FILENO, buffer, nbytes);
        close(fd[0]);
        break;
}
return 0;
}
```